Modulo 4 · Lezione 9: Lab, skill e MCP nel coding agent e nell'assistente

Un lab con un compito solo dell'ufficio del personale di Valtesa: preparare l'ingresso dei nuovi assunti. La procedura sta in una skill, le date le fa il codice vostro, i task e le spunte stanno nell'app che avete collegato la lezione scorsa (Trello, Notion o Airtable). Prima con Claude Code, poi con l'assistente, che diventa la versione 1.0; in fondo, se c'è tempo, di nuovo con Claude Code, per sapere a che punto sono gli ingressi.

Preparare la cartella

All'inizio del lab, dalla radice della vostra copia della repo del corso (la cartella che contiene `modulo4-agenti/`):

```
git switch main
git pull
git switch -c l09-lavoro
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro della lezione scorsa: `git add .` e `git commit -m "lavoro della lezione scorsa"`.

Preparare l'ambiente

Sempre dalla radice della repo:

```
cd modulo4-agenti/l09_lab-skill-mcp/scripts/working
source ../../../.venv/bin/activate
cp ../../../l08_mcp/scripts/working/.env .env
pytest
```

Su PowerShell il venv si attiva con `..\..\..\.venv\Scripts\Activate.ps1` e la chiave si copia con `copy ..\..\..\l08_mcp\scripts\working\.env .env`. Se la cartella della lezione scorsa non ha il `.env`: copiate `.env.example` in `.env` e incollate la chiave del corso dopo `OPENAI_API_KEY=`.

`pytest` deve dire `93 passed` (su Windows senza i permessi per i link simbolici: `92 passed, 1 skipped`). Niente da installare. Tenete aperto questo terminale, con il venv attivo: i comandi `python` della lezione si lanciano qui, mentre Claude Code lavora nella sua sessione.

Poi Claude Code, aperto nella cartella `working/` (nell'app desktop: una sessione Local, senza worktree), con Sonnet 5.5 a effort medio o alto (modello ed effort si scelgono con `/model`), e in **Manual**: dal selettore accanto al tasto di invio, o da terminale con Shift+Tab. Chiedete:

> quali connettori hai in questa sessione?

Con Claude Code va bene anche un altro connettore adatto che avete già; l'assistente dell'obiettivo 2 sa usare solo Trello, Notion e Airtable. Nella risposta ci deve essere il vostro: il connettore l'avete aggiunto su claude.ai la lezione scorsa e resta lì. Nell'app di Claude, se manca, aggiungetelo alla sessione dal pulsante **+** accanto al campo di testo. Se non avete ancora un account o il connettore: il passo 1 dell'esercizio della lezione scorsa (account gratuito, poi su claude.ai **Impostazioni → Connettori → Sfoglia**, l'app, **Connetti**).

I file della lezione

File	A cosa serve
`working/`	il progetto Valtesa: la soluzione del docente di Modulo 4 · Lezione 8, con i tre programmi dell'assistente già uniti in `assistente.py` (la versione 0.9, in `CHANGELOG.md`)
`working/dati/assunzioni.csv`	i sei nuovi assunti; una copia è in `ufficio-completo/`, dove la legge l'assistente
`working/ufficio-completo/procedura-ingresso-dotazioni.pdf`	la procedura di ingresso PR-PER-04
`working/specifica-skill.md`	la specifica della skill degli obiettivi 1 e 3: la richiesta dell'ufficio e i fatti; il resto lo scrivete voi
`working/specifica-assistente.md`	la specifica dell'assistente 1.0, obiettivo 2: la richiesta e i fatti; il resto lo scrivete voi
`ripartenza/1-skill/`, `2-assistente/`, `3-solleciti/`	i punti di ripartenza, uno per obiettivo

L'ingresso di un nuovo assunto

Quando arriva un nuovo assunto, l'Ufficio del Personale segue la procedura PR-PER-04. Chi fa cosa, e quando:

Quando	Cosa	Chi
10 giorni lavorativi prima	lettera di benvenuto, con data, ora e sede, referente e documenti da portare	Ufficio del Personale
10 giorni lavorativi prima	richiesta delle dotazioni nel portale	Responsabile di reparto
prima del primo giorno	visita medica preventiva, solo Produzione e Logistica	il medico competente, fissata dall'Ufficio del Personale
il primo giorno	badge, con l'account (art. 6)	Ufficio del Personale
il primo giorno	accoglienza: contratto, informative, regolamenti	Ufficio del Personale
il primo giorno	account e dotazioni del ruolo	IT
il primo giorno	dispositivi di protezione, solo Produzione e Logistica	Servizio di prevenzione e protezione
il primo giorno	i documenti	il nuovo assunto

Quando	Cosa	Chi
il primo giorno	presentazione del gruppo e piano di affiancamento	Responsabile di reparto
entro 30, 60, 90 giorni	i corsi obbligatori	Ufficio del Personale, iscrizione
fine della quarta settimana	colloquio di verifica	Responsabile di reparto
entro 6 mesi	scelta sulla destinazione del TFR	il nuovo assunto

Quali attività servono dipende da reparto, contratto e casi particolari: la procedura vale solo in parte per gli stagisti (art. 2), i corsi cambiano con il rischio del reparto e con gli attestati che uno ha già (art. 7), le dotazioni con il ruolo (art. 6). Le date prima dell'ingresso sono in giorni lavorativi, senza le festività; i corsi si contano in giorni di calendario.

La procedura in una skill, i conti nel codice, i task in Trello

	La procedura	I conti: date e indirizzo	I task e le spunte
Dove sta	una skill	`scadenze.py`, codice vostro	l'app: nei file non ci sono
Claude Code	la apre con `Skill`	lancia `scadenze.py` con `Bash`	i tool del connettore
L'assistente	la apre con `apri_skill`	un function tool, con lo stesso codice	`MCPServerStreamableHttp`, con il login

È la divisione delle ultime due lezioni. Una procedura che si ripete sta in una skill (Modulo 4 · Lezione 7). I conti sono codice vostro: uno script per il coding agent, un function tool per l'assistente (Modulo 4 · Lezione 8). Il lavoro di tutti sta nell'app dell'ufficio, e ci si arriva con il suo server MCP: per scriverci, e per leggere quello che hanno fatto gli altri.

I nuovi assunti

`dati/assunzioni.csv`, persone e date inventate:

Chi entra, e quando	Reparto, sede, contratto	Cosa ha di diverso
201 Elena Fabbri, 16/11/2026	Commerciale, Brescia	trasferte frequenti
202 Nicola Ferraresi, 16/11/2026	Produzione, Padova	ha già un attestato di sicurezza
203 Omar Benali, 01/12/2026	Logistica, Padova	userà il carrello elevatore
204 Chiara Galli, 01/12/2026	IT, Brescia, stage	è una stagista
205 Sara Conti, 14/12/2026	Amministrazione, Brescia	in azienda ci sono già due Sara Conti
206 Davide Russo, 11/01/2027	Produzione, Padova, capoturno	lo prepara l'assistente, nell'obiettivo 2

Lo spazio in Trello (o Notion, Airtable)

Lo spazio si chiama «Valtesa · nuovi ingressi», ed è nuovo: quelli della lezione scorsa non si toccano. Un task per persona, con le sue attività:

- **Trello**: una board, una lista «Ingressi», una card per persona con una checklist;
- **Notion**: una pagina, una sottopagina per persona con le caselle da spuntare;
- **Airtable**: una base, una tabella «Attività» con un record per attività e una casella Fatto.

Il titolo del task dice chi e quando, per esempio «Elena Fabbri · ingresso 16/11/2026»: è quello che permette di non preparare due volte lo stesso ingresso. In Airtable, dove le righe sono le attività, il task di una persona sono le sue righe, e il titolo sta nel campo Persona. Lo spazio lo crea la skill la prima volta. Il resto del formato lo decidete voi nella specifica.

In Manual Claude Code chiede il permesso per ogni tool dell'app, anche per quelli che leggono. A quelli che leggono o creano rispondete sì; a un tool che modifica o cancella quello che c'era, no. Spesso è lo stesso tool: in Trello `trelloWriteCard` crea, modifica e archivia. Lo dice l'argomento `action` nella domanda di Claude Code: `create` e `add_item` creano; `update`, `update_item`, `move`, `archive` cambiano quello che c'era. Per il tool che aggiunge le attività (con Trello `trelloWriteChecklist`) le domande sono decine: potete scegliere di non farvi chiedere più, sapendo che il permesso vale per tutto il tool, che sa anche spuntare e rinominare, su tutto il vostro account, e che resta salvato in `.claude/settings.local.json`. Per questo si lavora in uno spazio nuovo, e nell'obiettivo 3, che legge soltanto, guardate voi nella sessione che il tool non compaia. A fine lab, se volete, togliete quella riga da `.claude/settings.local.json`.

Con Notion o Airtable, nelle richieste e nei comandi di questa dispensa, al posto di «Trello» e di `trello` scrivete la vostra app.

Obiettivo 1: l'ingresso di un nuovo assunto in una skill (60')

File: `working/specifica-skill.md`, poi `working/.claude/skills/ingresso-nuovo-assunto/` e `working/scadenze.py`, che all'inizio non ci sono: li crea il coding agent.

1. **Specificare.** Aprite `specifica-skill.md`: in testa ci sono la richiesta dell'ufficio e i fatti, gli stessi della slide. Il resto lo scrivete voi, leggendo la procedura (`ufficio-completo/procedura-ingresso-dotazioni.pdf`, tre pagine):

   - **La procedura**: i passi che la skill fa seguire, in ordine, e i casi a cui stare attenti (la slide dei nuovi assunti ne ha cinque);
   - **Lo script**: cosa stampa `scadenze.py` per un nuovo assunto, con `--tutti`, che data dare a quello che la procedura lascia aperto (la visita medica, il colloquio, i corsi «prima della mansione»), e cosa provano i suoi test;
   - **La descrizione**: una o due frasi su una riga, cosa fa la skill e quando usarla, con le parole con cui chiederete di preparare un ingresso; senza «parola: testo», perché l'intestazione è YAML e i due punti seguiti da uno spazio la rompono (Modulo 4 · Lezione 7);
   - **Il formato in Trello** (o Notion, Airtable): dove stanno gli ingressi, com'è fatto il task di una persona, com'è scritta un'attività, come si riconosce un ingresso già preparato;

- **Criteri di accettazione**: da tre a cinque, ognuno verificabile guardando la sessione, Trello, o lanciando un comando.

La tabella delle attività non la scrivete: la ricava il coding agent dal PDF, e voi la controllate al passo 3.

2. **Costruire.** Una sessione nuova di Claude Code in `working/`:

   > Leggi specifica-skill.md e crea la skill ingresso-nuovo-assunto come dice la specifica: SKILL.md con la mia descrizione nel campo description, la procedura e la tabella delle attività che leggi nella procedura PR-PER-04, e scadenze.py con i suoi test. Non preparare ancora nessun ingresso e non usare Trello. Prima dimmi il piano.

   (con Notion o Airtable, al posto di Trello il nome della vostra app). Se il piano vi fa una domanda, rispondete voi; poi «Va bene, procedi.». Alla fine lanciate voi `pytest`: deve essere verde.

3. **Controllare.** Aprite `.claude/skills/ingresso-nuovo-assunto/SKILL.md` e la procedura, e guardate la tabella delle attività su tre casi: cosa fa con la stagista (art. 2), con chi ha già l'attestato di formazione generale (art. 7), con chi usa il carrello (artt. 5 e 7). Se una riga è sbagliata, correggetela, o chiedetelo al coding agent (i numeri giusti sono nei valori di controllo, in fondo all'obiettivo). Poi lanciate voi:

   ```
   python scadenze.py 205
   ```

   Poi un commit, così la ripartenza non perde il vostro lavoro: `git add .` e `git commit -m "skill ingresso-nuovo-assunto"`.

4. **Usare.** Una sessione nuova (`/clear`, o una finestra nuova), così il coding agent parte con l'elenco delle skill aggiornato. Senza nominare la skill:

   > Preparami in Trello l'ingresso di Sara Conti.

   Poi, nella stessa sessione:

   > Prepara in Trello gli ingressi di novembre e dicembre dei nuovi assunti di dati/assunzioni.csv.

   Il secondo comando non deve rifare Sara Conti: è la prima prova che la skill non prepara due volte lo stesso ingresso.

   Nella sessione guardate: `Skill(ingresso-nuovo-assunto)`, lo script, i tool dell'app; prima quelli che leggono, poi quelli che scrivono. In Trello (o Notion, Airtable) guardate i task mentre arrivano, e confrontate le date con `python scadenze.py --tutti`.

5. **Rilanciare.** Un'altra sessione nuova, la stessa richiesta di novembre e dicembre. Non deve scrivere niente nell'app. Se scrive, fermatelo (Esc) e segnatevi cosa ha scritto: il perché lo vediamo al rientro, e i doppioni si tolgono prima dell'obiettivo 2.

Fatto quando nella sessione c'è `Skill(ingresso-nuovo-assunto)`; i cinque task di novembre e dicembre hanno le attività del loro ruolo e le date di `scadenze.py`; il secondo lancio non scrive niente; in `specifica-skill.md`, nella sezione «Cosa ha deciso le attività», due righe: per una persona, cosa nella skill ha deciso le sue attività. Poi un commit.

Valori di controllo. `python scadenze.py 205` dice `sara.conti3@valtesa.it` e, per la lettera, venerdì 27/11/2026. Con la skill del docente le attività sono 63: Elena Fabbri 13, Nicola Ferraresi 14, Omar Benali 16, Chiara Galli 7, Sara Conti 13. Con la vostra il numero può cambiare (per esempio se il badge sta nella stessa riga dell'account); contano i casi: la stagista senza lettera, documenti, accoglienza, presentazione del gruppo e affiancamento, colloquio e TFR (l'art. 2 le applica solo account, dotazioni e formazione); Nicola Ferraresi senza il corso di formazione generale; Omar Benali con la patente tra i documenti e il corso carrelli; visita medica e protezioni solo per Nicola Ferraresi e Omar Benali.

Se vi fermate. Dalla cartella `working/`:

```
cp -r ../ripartenza/1-skill/. .
```

Su PowerShell: `Copy-Item -Recurse -Force ..\ripartenza\1-skill\* .`. Copia la skill del docente (per Trello, Notion e Airtable), `scadenze.py` e il suo test, sopra i vostri. Poi ripartite dal passo 4. Se nell'app ci sono già task fatti a metà, archiviateli a mano prima.

Estensioni. La stessa richiesta senza la skill: spostate la cartella `.claude/skills/ingresso-nuovo-assunto/` e `scadenze.py` fuori da `working/` (come nella nostra misura, dove lo script non c'era), chiedete gli ingressi in una board con un altro nome, due volte, e confrontate i task. Poi rimettete tutto dov'era.

Rientro: cosa ha cambiato la skill (da leggere dopo l'obiettivo 1)

La stessa richiesta, «Prepara in Trello gli ingressi …», con e senza la skill del docente (Claude Code 2.1.289, Sonnet 5.5 a effort medio, una sessione per riga, 21/10/2026):

	Senza la skill	**Con la skill**
Le attività di Chiara Galli, la stagista	10, con «da confermare con il Personale»	7, come dice la tabella
Il formato delle attività	libero, diverso da un lancio all'altro	«attività · chi la fa · entro», uguale per tutti
Le date e l'indirizzo di Sara Conti	giusti	giusti, da `scadenze.py`
Il secondo lancio	una seconda checklist su ogni card, 36 attività in più	nessuna scrittura
Il primo lancio, tempo e costo a prezzo di API	161 secondi, 0,68 $	135 secondi, 0,71 $

Senza la skill Sonnet legge la procedura e conta i giorni lavorativi senza sbagliare: il PDF sono tre pagine. La skill non lo rende più bravo né più economico: rende il lavoro uguale ogni volta. Le stesse regole per i casi dubbi (la stagista), lo stesso formato, che poi leggono la seconda parte della skill e l'assistente, e nessun ingresso fatto due volte.

Perché senza la skill ha scritto due volte. La lettura delle card di Trello (`trelloReadCard`) risponde `"checklists": []` per ogni card, anche quando la checklist c'è; le checklist si leggono con un altro tool, `trelloReadChecklist`. Senza la skill il modello ha creduto alla prima risposta e ha aggiunto una seconda checklist a ogni card. Con una skill che non dice come leggere le checklist, al rilancio non ha scritto niente, ma nel riepilogo ha scritto «nessuna delle cinque card ha la checklist», che era falso. Una skill che lo dice ha tre righe in più: per sapere se un ingresso c'è già basta il titolo; nel riepilogo non si dice niente di quello che non si è letto; le attività di una card si leggono dalle sue checklist. Con queste righe non ha scritto niente, e nel riepilogo ha detto che le checklist non le aveva lette. Il modello lavora con quello che i tool gli restituiscono: se un tool risponde in modo fuorviante, quale tool usare lo deve dire la skill.

Il secondo lancio senza skill ha scritto 36 attività e non 61 perché la seconda checklist era in un formato più corto, con i corsi in una riga sola: anche il formato cambia da un lancio all'altro.

Prima dell'obiettivo 2. Se il vostro rilancio ha scritto doppioni, toglieteli a mano dall'app (in Trello: la seconda checklist di ogni card). E aggiungete alla vostra skill, al passo che legge l'app, le tre righe che mancano: per sapere se un ingresso c'è già basta il titolo; nel riepilogo non si dice niente di quello che non si è letto; le attività di una card si leggono dalle sue checklist. Potete chiederlo a Claude Code: «Aggiungi alla skill ingresso-nuovo-assunto, al passo che legge Trello, queste tre regole: …». Servono anche all'assistente: la sua skill si scrive dalla vostra.

Obiettivo 2: l'assistente dell'ufficio, versione 1.0 (60')

File: `working/specifica-assistente.md`, poi `assistente.py`, `skill-assistente/ingresso-nuovo-assunto/SKILL.md`, `CHANGELOG.md`.

L'ufficio non usa Claude Code: usa l'assistente. `assistente.py` è già un programma solo, la versione 0.9: i tre programmi della lezione scorsa (`assistente.py`, `skill_agente.py`, `app_agente.py`) li abbiamo uniti noi, ed è scritto in `CHANGELOG.md`; `skill.py` e `app.py` tengono il codice delle skill e dell'app, senza agente. Per preparare un ingresso all'assistente mancano le date e la procedura: la versione 1.0 le aggiunge, e come lo decidete voi, nella specifica.

Passando da Claude Code all'assistente, alcune cose restano e altre no:

	Claude Code	**L'assistente**
La procedura	la skill, aperta con `Skill`	la stessa procedura, riscritta per i suoi tool, aperta con `apri_skill`
I conti	`python scadenze.py`, con `Bash`	lo stesso codice, in un function tool
Trello	il connettore di claude.ai	un login suo, in questa cartella; solo i tool del `tool_filter`

	Claude Code	L'assistente
I permessi	in Manual chiede prima dei tool di Trello, anche di quelli che leggono	non chiede niente: i tool che riceve li usa

La board resta la stessa: l'assistente legge i task che ha scritto Claude Code e ne aggiunge uno.

1. **Spuntare.** In Trello, a mano, come se l'avessero fatto il Personale, l'IT e i responsabili, segnate come fatte cinque attività (se la vostra skill ha unito o diviso queste righe, spuntate quelle che corrispondono):

   - Elena Fabbri: la lettera di benvenuto, la richiesta delle dotazioni, l'account e le dotazioni;
   - Nicola Ferraresi: la lettera di benvenuto, la visita medica.

   Le spunte servono qui, alla domanda di Nicola Ferraresi, e nell'obiettivo 3.

2. **Login.** Nel terminale, in `working/` con il venv attivo:

   ```
   python assistente.py --app trello --login
   ```

   (o `notion`, `airtable`). Si apre il browser: entrate e acconsentite. Il login è fatto quando il terminale scrive «Login a trello fatto». Il token della lezione scorsa è rimasto in quella cartella: qui il login si rifà.

3. **Specificare.** Aprite `specifica-assistente.md`: in testa la richiesta e i fatti, gli stessi della slide. Scrivete voi:

   - **Cosa aggiungere**: cosa serve all'assistente per le date e per la procedura, in quale file e con quale nome. L'assistente non ha `Bash` né `Skill`: lo script e la skill dell'obiettivo 1 così non li può usare;
   - **Cosa non cambia**: quello che deve restare com'è, perché l'assistente di sempre non si rompa;
   - **Criteri di accettazione**: da tre a cinque, verificabili con `pytest`, con un comando o guardando la traccia.

4. **Costruire.** Una sessione nuova di Claude Code:

   > Leggi specifica-assistente.md e fai quello che dice. Prima dimmi il piano.

   Leggete il diff: `git add -A`, poi `git diff --staged`, che mostra anche la skill nuova (si esce con `q`). Poi `pytest`, verde, e:

   ```
   python assistente.py --app trello --istruzioni
   python assistente.py --app trello --tool
   ```

   Il primo stampa le istruzioni: in fondo la riga di Trello e l'indice con le due skill. Il secondo stampa i tool del server; con `*` quelli che riceve il modello. Rileggete la skill dell'assistente: le date con il vostro function tool, i file con `leggi_file`, la parte su chi entra.

5. **L'ufficio.** Nel terminale:

```
python assistente.py --app trello "Prepara in Trello l'ingresso di Davide Russo."
python assistente.py --app trello "Prepara in Trello tutti gli ingressi dei nuovi assunti."
```

Il primo prepara il sesto task; il secondo non deve creare niente: i cinque di Claude Code e quello di Davide Russo ci sono già. Per circa un minuto non stampa niente: lavora. Nella traccia cercate `apri_skill`, il vostro tool per le date, i tool di Trello che leggono, poi quelli che scrivono.

6. **Chi entra.**

```
python assistente.py --app trello "Sono Nicola Ferraresi, entro il 16 novembre: cosa devo portare il primo giorno, e cosa è già pronto?"
python valuta.py --solo 1,2,3,4,7,9,10
```

`valuta.py` fa all'assistente le domande sui documenti di Modulo 4 · Lezione 6 (la 5, la 6 e la 8 sono conti sul CSV, che con la sola lettura non tornano): con le skill nelle istruzioni devono restare giuste. Se una è sbagliata, guardate nella sua traccia se l'assistente ha aperto la skill degli ingressi su una domanda dei documenti: allora è la descrizione della skill da stringere. Poi la voce 1.0 in `CHANGELOG.md`, se il coding agent non l'ha già scritta, e un commit.

Fatto quando nella traccia ci sono `apri_skill`, il vostro tool per le date e poi i tool di Trello; Davide Russo ha il suo task, con le attività del suo ruolo, e gli altri cinque non sono rifatti; la risposta a Nicola Ferraresi dice cosa portare (anche l'attestato) e cosa è già spuntato nel suo task; `valuta.py` dà 7 su 7; `pytest` è verde; la voce 1.0 è in `CHANGELOG.md`.

Valori di controllo. Con la skill del docente Davide Russo ha 15 attività, con la lettera entro il 23/12/2026: i dieci giorni lavorativi prima dell'11 gennaio passano su Natale, Capodanno ed Epifania. Come capoturno ha portatile e cellulare, oltre alle postazioni del reparto.

Se vi fermate. Dalla cartella `working/`:

```
cp -r ../ripartenza/2-assistente/. .
```

Su PowerShell: `Copy-Item -Recurse -Force ..\ripartenza\2-assistente\* .`. È l'assistente 1.0 del docente (`scadenze_ingresso`, i test, la skill dell'assistente, `CHANGELOG.md`), con la skill del coding agent e `scadenze.py` dell'obiettivo 1: si copia sopra i vostri. Se non li avete fatti, spunte e login dei passi 1 e 2; poi ripartite dal passo 5. La skill del docente riconosce i task dal titolo «Nome Cognome · ingresso GG/MM/AAAA»: se le vostre si chiamano in un altro modo, rinominatele in Trello.

Se qualcosa non va.

Cosa vedete	Cosa fare
Il browser dice che `localhost` rifiuta la connessione	rilanciate il login: `python assistente.py --app trello --login`
«La porta 8765 è occupata»	un altro login è aperto in un altro terminale: chiudetelo (Ctrl+C) e rilanciate

Cosa vedete	Cosa fare
Mentre l'assistente lavora si apre il browser, o errori 401	il token non vale più: Ctrl+C, poi `python assistente.py --app trello --login`
La risposta dice che ha preparato solo una parte delle attività	il task a metà non lo finisce, perché per la skill basta il titolo: archiviatelo in Trello e rilanciate la stessa richiesta
L'assistente risponde che non può rispondere	la riga di Trello (`ISTRUZIONI_APP`) non è nelle istruzioni: guardate `--istruzioni`
`ModuleNotFoundError: No module named 'agents'`	il venv non è attivo: `source ../../../.venv/bin/activate`

Estensioni. L'unione dei tre programmi rifatta da voi: in una copia della cartella della lezione scorsa, con il coding agent, e poi il confronto con la 0.9. `require_approval` sul tool di Trello che aggiunge le attività: è un parametro del server MCP nell'SDK, e per i tool che indicate il giro si ferma e chiede un sì, come `needs_approval` di Modulo 4 · Lezione 5. Oggi `rispondi()` passa da `esegui_con_approvazione` solo con `ESEGUI_PYTHON`: con l'approvazione su Trello va fatto passare anche il giro con l'app, altrimenti si ferma senza risposta. Chi entra vede solo il suo task: provate prima con una riga nelle istruzioni, poi chiedetevi cosa serve nel codice perché sia una regola e non un consiglio. Un `README.md` per l'ufficio: come si lancia l'assistente e cosa sa fare.

Rientro: la stessa procedura in due harness

	Claude Code	L'assistente
La procedura	`Skill(ingresso-nuovo-assunto)`	`apri_skill("ingresso-nuovo-assunto")`
I conti	`python scadenze.py`, con `Bash`	un function tool, nel processo dell'assistente (il nostro è `scadenze_ingresso`)
I task in Trello	`trelloWriteChecklist`, eseguito dal server di Trello	lo stesso tool, dallo stesso server

La skill e il codice sono vostri e servono a tutti e due: la skill in due copie, una per gli strumenti di ciascuno, lo script uguale. Trello è dell'ufficio, e tutti e due ci arrivano con il suo server MCP.

Obiettivo 3: a che punto sono gli ingressi, letto da Trello (40')

File: la sezione «Seconda parte» di `specifica-skill.md`, la skill, `scadenze.py`.

Si torna a Claude Code. Le spunte sono quelle dell'obiettivo 2 (passo 1): se non le avete messe, mettetele adesso. Se si è in ritardo, questo obiettivo si fa a casa.

1. **Specificare.** Nella sezione «Seconda parte» di `specifica-skill.md` ci sono la richiesta dell'ufficio e i fatti, gli stessi della slide. Sotto scrivete voi cosa entra nel file e cosa no (le attività spuntate, quelle già scadute, quelle senza una data), come si allarga la descrizione della skill (altrimenti «a che punto sono gli ingressi?» non la apre), e due o tre criteri di accettazione, da controllare confrontando il file dei solleciti con Trello.

2. **Costruire.** Una sessione nuova:

   > Leggi la sezione «Seconda parte» di specifica-skill.md e aggiungi alla skill ingresso-nuovo-assunto la seconda parte, con la descrizione allargata e --entro in scadenze.py con il suo test. Non usare ancora Trello. Prima dimmi il piano.

   Alla fine `pytest`, verde, e `python scadenze.py --entro 2026-11-09`, che deve dire lunedì 16/11/2026.

3. **Chiedere.** Un'altra sessione nuova:

   > A che punto sono gli ingressi al 9 novembre 2026?

   Questa parte legge soltanto: nella sessione non deve comparire nessun tool di Trello che scrive (`trelloWriteChecklist` o `trelloWriteCard`). Se avete dato il permesso fisso al tool delle checklist non vi chiede niente: guardate voi, e se compare, fermatelo con Esc.

4. **Confrontare.** Aprite `output/solleciti-2026-11-09.md` e Trello, e controllate task per task: le attività spuntate non devono esserci, quelle degli ingressi di dicembre e di Davide Russo nemmeno (la prima scadenza di dicembre è il 17/11, per Sara Conti il 27/11; per Davide Russo il 23/12).

Fatto quando nella sessione ci sono solo tool di Trello che leggono; `output/solleciti-2026-11-09.md`, diviso per responsabile, passa i vostri criteri: confrontato con Trello, niente di spuntato, niente con una data dopo il 16/11/2026 (il 9 novembre più 5 giorni lavorativi); le attività senza data come dicono i vostri criteri. Poi un commit (`output/` resta fuori: git la ignora).

Valori di controllo. Con le spunte dell'obiettivo 2 e la skill del docente, che mette nel file le attività senza data di chi entra entro il 16/11, il file ha 12 attività di Elena Fabbri e Nicola Ferraresi, una sola scaduta: la richiesta delle dotazioni di Nicola Ferraresi, entro il 02/11/2026. C'è anche il suo corso di sicurezza a rischio alto, che non ha una data («prima della mansione») ed entra perché il suo ingresso cade entro il 16/11.

Se vi fermate. Dalla cartella `working/`:

```
cp -r ../ripartenza/3-solleciti/. .
```

Su PowerShell: `Copy-Item -Recurse -Force ..\ripartenza\3-solleciti\* .`. Poi le spunte, se non le avete messe, e ripartite dal passo 3. Copia anche `scadenze.py`: se poi `pytest` è rosso sui test dell'assistente, il vostro function tool usava funzioni che lo script del docente non ha; adattatelo, o chiedetelo a Claude Code. La skill del docente cerca la board «Valtesa · nuovi ingressi» e riconosce i task dal titolo «Nome Cognome · ingresso GG/MM/AAAA»: se le vostre si chiamano in un altro modo, rinominatele in Trello.

Estensioni. Con il connettore Gmail di claude.ai, se lo avete: «Prepara una bozza di mail per ogni responsabile, con i suoi solleciti di output/solleciti-2026-11-09.md.» Solo bozze: se chiede di inviare, no. Con il connettore Google Calendar: «Metti in calendario il primo giorno di Davide Russo, da dati/assunzioni.csv.»

Dopo questa lezione

La prossima lezione è Modulo 4 · Lezione 10: più agenti nel coding agent, sub-agent con ruoli diversi che collaborano.

Prima della prossima lezione: un commit del vostro lavoro sul branch `l09-lavoro`, poi dalla radice della repo:

```
git switch main
git pull
```